

# DOCUMENTATION TECHNIQUE

**Projet :** Application Mobile de Gestion de Notes (*To-Do List*)

**Cadre :** Validation de la Semaine 6 - Formation DCLIC (OIF)

**Technologie :** Flutter SDK & SQLite (Bases de données relationnelles locales)

**Niveau :** Intermédiaire

## 1. Introduction et Objectifs du Projet

Ce document constitue le rapport technique officiel du projet de fin de module de la semaine 6. L'objectif principal était de concevoir et développer une application mobile de prise de notes robuste, performante et éco-conçue en utilisant le framework **Flutter**.

Le projet valide l'acquisition des compétences clés suivantes :

- La maîtrise du cycle de vie d'une application (écrans de démarrage et transitions).
- L'interfaçage graphique avancé respectant des wireframes stricts.
- L'implémentation d'une persistance de données locale sécurisée via un moteur **SQLite**.
- La gestion de la navigation et des flux d'authentification utilisateur.

## 2. Architecture Technique du Projet

L'application s'appuie sur une architecture de type modulaire inspirée du modèle MVC (Modèle-Vue-Contrôleur), garantissant une séparation claire entre la logique métier, les données et l'interface utilisateur.

```
lib/
├── database/
│   └── database_manager.dart    # Service de persistance et gestion des
requêtes SQL
├── model/
│   ├── user.dart               # Entité et adaptateurs pour la table "users"
│   └── todo_list.dart          # Entité et adaptateurs pour la table
"todo_list"
└── views/
    ├── splash_screen.dart      # Interface asynchrone de pré-chargement
    ├── login_interface.dart    # Formulaire d'authentification et gestion
des saisies
    └── todo_list_interface.dart# Écran CRUD principal de gestion des notes
```

### 3. Modélisation et Persistance des Données (SQLite)

Le stockage des données est confié au package officiel `sqflite`. La base de données locale est initialisée sous le nom `todo_list.db`.

#### 3.1 Schéma Relationnel des Tables

Le système déploie deux tables distinctes lors de sa première exécution :

**Table `users` :**

- `id` : INTEGER PRIMARY KEY AUTOINCREMENT
- `email` : TEXT (Non null)
- `password` : TEXT (Non null)

**Table `todo_list` :**

- `id` : INTEGER PRIMARY KEY AUTOINCREMENT
- `tache` : TEXT (Non null)

#### 3.2 Sécurisation du Démarrage (*Data Seeding*)

Afin de répondre à la contrainte d'authentification sans écran d'inscription préalable, un utilisateur administrateur unique est injecté automatiquement via le code lors de la création de la base de données (`onCreate`) :

- **Identifiants** : `admin@gmail.com / password1234`
- **Contrôle des conflits** : L'instruction utilise `ConflictAlgorithm.ignore` pour empêcher toute tentative de duplication ou de corruption de données lors des démarrages ultérieurs de l'application.

### 4. Implémentation de l'Interface Utilisateur (UI/UX)

Le développement des interfaces graphiques s'est concentré sur le respect rigoureux des wireframes fournis et sur la résolution de contraintes ergonomiques mobiles majeures.

#### 4.1 Gestion du Double Splash Screen

Afin d'éviter l'effet visuel indésirable de "double chargement" (le splash natif du système suivi du splash de l'application), la configuration a été harmonisée :

- Le splash natif est configuré en blanc uni (`#FFFFFF`) via le fichier `pubspec.yaml`.

- Dès que le moteur Flutter s'éveille, le widget `SplashScreen` prend le relais de manière transparente en affichant les bandeaux indigo de l'OIF et le logo arrondi à l'aide d'un composant `ClipRRect(borderRadius: BorderRadius.circular(30))`.

## 4.2 Optimisation du Formulaire de Connexion

L'écran `LoginInterface` résout deux problématiques courantes en développement mobile :

- **Élimination du *Keyboard Overflow*** : L'arborescence utilise un widget `SingleChildScrollView` pour rendre le formulaire défilant. L'affichage s'adapte dynamiquement lors du déploiement du clavier virtuel, évitant ainsi le crash visuel des bandes hachées jaunes et noires.
- **Gestion des Erreurs Déportées** : Pour conserver l'esthétique des ombres portées basses sans déformer les champs, la validation classique de Flutter a été remplacée par un suivi d'état manuel (`_emailError` et `_passwordError`). Les messages d'affichent à l'extérieur des conteneurs, préservant l'alignement du texte saisi.

## 4.3 Ombres Portées Asymétriques

Conformément aux maquettes, les champs de texte, les boutons et les cartes de la liste des tâches adoptent un relief plat et moderne. Cela a été matérialisé en isolant les éléments dans des widgets `Container` décorés de `BoxShadow` configurés avec un décalage géométrique vertical exclusif :

```
offset: const Offset(0, 4) // Projection uniquement vers le bas, 0 pixel sur les côtés
```

## 5. Gestion des Flux et Sécurité de Navigation

L'application applique une politique stricte de gestion de la pile d'écrans pour sécuriser les données :

- **Transition Login → Liste** : Lors d'une authentification réussie, la transition s'effectue via `Navigator.pushReplacement`. L'écran de connexion est instantanément détruit de la mémoire. Un utilisateur connecté ne peut donc pas revenir en arrière sur le formulaire en utilisant le bouton physique de son téléphone.

- **Déconnexion (*Logout*)** : L'action de déconnexion réapplique cette méthode de remplacement pour vider l'historique et verrouiller à nouveau l'accès à l'interface `ToDoListInterface`.

## 6. Guide d'Exploitation et Commandes Terminal

Pour assurer la maintenance ou l'évaluation du projet, les commandes suivantes doivent être exécutées dans l'ordre au sein du terminal de développement :

- **Nettoyage complet du projet et des builds natifs :**  
`flutter clean`
- **Résolution et téléchargement des dépendances :**  
`flutter pub get`
- **Génération automatique des icônes d'application (Launcher Icons) :**  
`dart run flutter_launcher_icons`
- **Exécution de l'application en mode débogage :**  
`flutter run`

## 8. Conclusion et Perspectives

L'application de gestion de notes développée répond en tous points aux exigences fonctionnelles et esthétiques du cahier des charges de la semaine 6. L'intégration d'une base de données relationnelle locale couplée à une interface utilisateur soignée et résistante aux bugs d'affichage (*overflow*) démontre une solide compréhension des fondamentaux de Flutter.

Pour une version future, il serait pertinent d'étendre ce travail en intégrant :

- Le chiffrement des mots de passe en base de données avec l'algorithme SHA-256.
- Une relation de clé étrangère (`user_id`) dans la table `todo_list` pour ouvrir l'application à un usage multi-utilisateurs.

## Ressources

### Wireframes et lien figma

Lien figma : <https://www.figma.com/design/MilySsiE6xFTPqEjWrFNWw/gestion-de-notes?node-id=0-1&t=IDmpOTpUWM2shM5H-1>

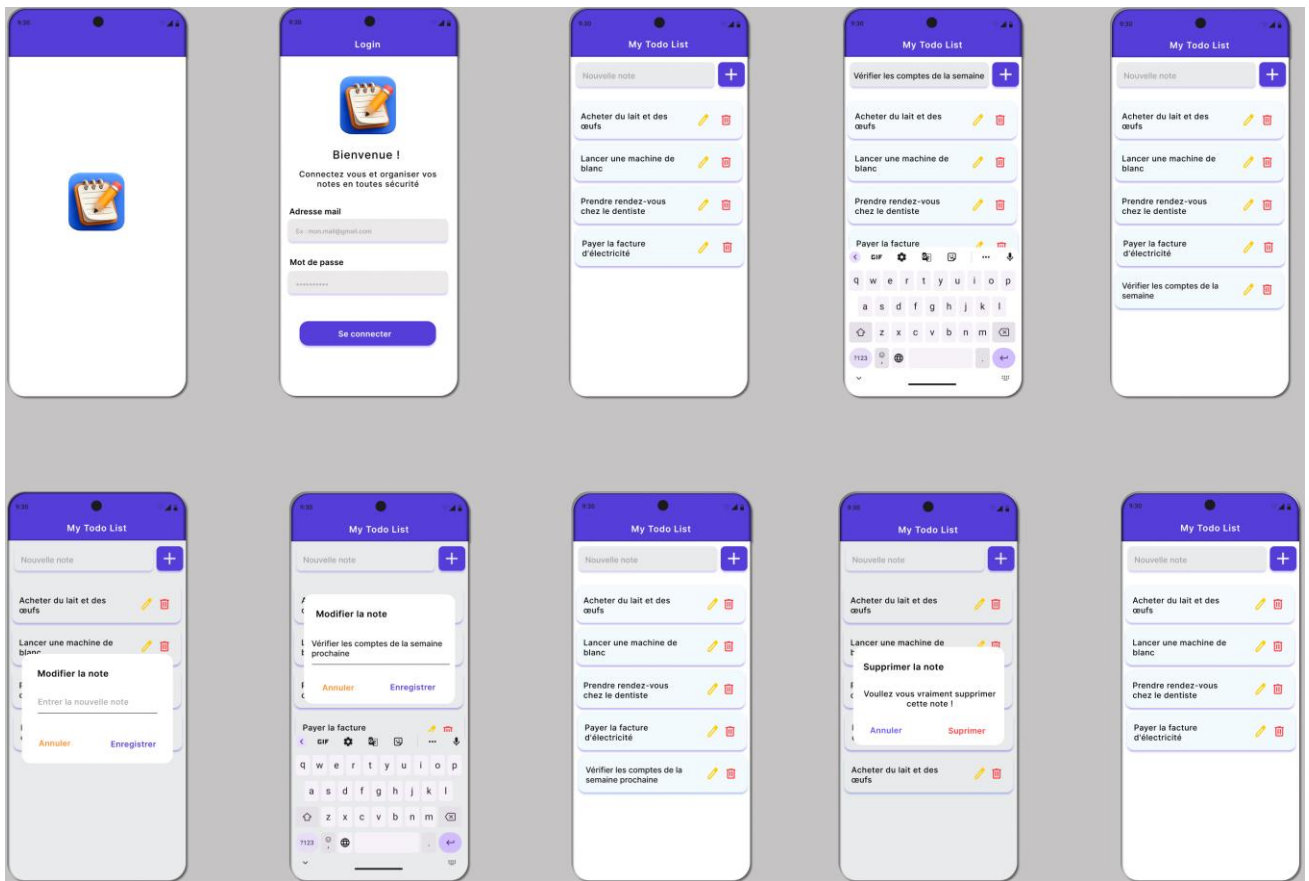


Figure 1 : wireframes app todo list